



Cairo University

Faculty of Engineering

Electronics and Communications Department



Project #3

**Matched Filters, Correlators, ISI,
and Raised Cosine Filters**

Course: Introduction to Communications

Supervised By: Dr. Mohamed Nafie

Eng. Mohamed Khaled

Academic Year: 2025 / 2026

Students Information:

Code	Sec	Name	#
91240202	1	باسل شريف حميد مصطفى	1
91240586	3	كيرلس سمير خليل كراس	2
91240651	3	محمد حمدي فتحي	3
91240645	3	محمد جمال جمعه السيد	4

2nd Year — Electronics and Communications

Contents

1. Introduction	3
2. System Model	3
2.1 PAM Signal Generation	3
2.2 Pulse Shaping Function	5
2.3 AWGN Channel Model	6
3. Requirement 1 — Matched Filter and Correlator (Noise-Free)	6
3.1 Matched Filter Design	6
3.2 Non-Matched Filter (Filter 2)	6
3.3 Sampling Instants	7
3.4 Correlator Receiver	7
3.5 Results — Requirement 1a: Filter Output Comparison	8
3.6	8
3.6 Results — Requirement 1b: Matched Filter vs. Correlator	10
4. Requirement 2 — Noise Analysis and BER	11
4.1 Simulation Setup	11
4.3 Theoretical BER	13
4.4 Results — BER vs E_b/N_0	14
5. Requirement 3 — ISI and Eye Patterns	15
5.1 Inter-Symbol Interference (ISI)	15
5.2 Square-Root Raised Cosine Filtering	15
5.3 Filter Parameters	16
5.4 Eye Pattern Interpretation	16
5.5 Results — Eye Patterns	17
6. MATLAB Implementation	25
7. Conclusion	26
8. References	26

1. Introduction

In digital communication systems, the reliable transmission of binary data over noisy channels requires careful design at both the transmitter and receiver. A fundamental challenge is the recovery of the original bit sequence from a received waveform that has been corrupted by Additive White Gaussian Noise (AWGN). The choice of receiver filter plays a decisive role in the quality of this recovery.

This project investigates three closely related topics: the design and performance of matched filters and correlator receivers, the statistical analysis of Bit Error Rate (BER) versus signal-to-noise ratio, and the effect of Inter-Symbol Interference (ISI) in bandlimited channels together with its mitigation using raised cosine pulse shaping.

Pulse Amplitude Modulation (PAM) is used as the transmission scheme throughout. The transmitted signal is modelled as a convolution of a train of polar binary impulses (± 1) with a pulse shaping function $p(t)$, and the resultant waveform passes through an AWGN channel before reaching the receiver. The project is implemented entirely in MATLAB, using only the functions explicitly permitted by the project specification: `upsample`, `conv`, `fliplr`, `downsample`, `randn`, `erfc`, `rcosine`, and `eyediagram`. Other functions used are user defined. In this report, we will introduce our source code explanation block by block

2. System Model

2.1 PAM Signal Generation

As shown in figure 1, the lines 4-6 are simply the definition of symbol duration and number of samples we want MATLAB to take per duration (`nspd`). Defining them separately from other variables depending on them allows us to examine different test cases other than the one used for our project as will be shown in this section. Line 6 introduces the time axis. It is a row vector of size `nspd` (5 here as required from project) beginning from 0, moving with a step of $T_s/nspd$ (200msec here). We can achieve more precision by changing the value of `nspd` in line 5, and the time axis will be resimulated as we need. If `nspd` is not defined independently, we would have

to recalculate the time axis or the pulse shaping signal ourselves for other test cases.

At line 8, we define the number of binary samples representing the message we want to transmit. Then transmitted PAM signal $y[n]$ is constructed by three steps. First, 10 random binary bits are generated using `randi` function as shown in line 9. We chose X (the message symbol) to be a random digital signal in order to verify that the code works properly for any input pattern. Then X is mapped to the polar sequence $\{+1, -1\}$ as shown in line 10 via the relation:

$$symbol = 2 \times bit - 1 \quad \dots (Eq. 1)$$

Reference: Proakis, J.G. & Salehi, M., "Digital Communications", 5th Ed., Eq. (1.1-6)

Second, the polar symbol sequence is upsampled by a factor of $L = 5$ (the number of samples per symbol period $T_s = 1$ s), inserting four zeros between consecutive symbols to represent the impulse train at a sampling rate of 5 Hz, and match the size of the pulse shaping function producing a row vector of size $(n_{spd} * \text{number of binary samples})$.

Third, the upsampled sequence is convolved with the discrete pulse shaping function $p[n]$ to produce the transmitted signal:

$$y[n] = \sum_k a_k \cdot p[n - k \cdot L] \quad \dots (Eq.2)$$

Reference: Haykin, S., "Communication Systems", 4th Ed., Section 6.1

```

1  %% Requirement 1
2  clc; clearvars; close all;
3
4  Ts = 1; %symbol duration
5  nspd = 5; %number of samples per duration
6  t = 0: Ts/nspd : (nspd-1)/nspd * Ts; %time axis
7
8  no_of_binary_samples = 10;
9  x = randi([0 1], 1, no_of_binary_samples);
10 bps = 2*x-1; %binary polar signal
11 bps = upsample(bps, nspd);
12
13 p = -5/Ts * t + 5; %pulse shaping function
14 p_normalized = normalize_power(p); %Renamed to avoid built-in conflict
15
16 y = conv(bps, p_normalized);
17
18 h_matched = fliplr(p_normalized); %Matched Filter
19 y_filtered1 = conv(y, h_matched);
20 y_filtered1_resampled = downsample(y_filtered1, nspd, nspd-1);

```

Figure 1 Matched Filter Code

2.2 Pulse Shaping Function

As shown in line 13, pulse shaping signal will generally be a row vector of the same size as t , which has the size defined in line 5, and values representing the pulse shaping function defined in our project. The advantage here of using independent time axis code instead of manually putting $[5 \ 4 \ 3 \ 2 \ 1]/\sqrt{55}$ is that if we need to simulate another pulse shaping pattern, or even the same sampling function with another precision, all what we need to is to replace the right-hand side in line 13 with the function of the new pulse we want. Line 14 takes the abnormalized pattern in line 13 and passes it through the function `normalize_power`.

The pulse shaping function used in our code is a linearly decreasing ramp defined over one symbol period $[0, T_s]$. In discrete form, sampled at 5 equally spaced points, the abnormalized pulse is $[5, 4, 3, 2, 1]$. To satisfy the unit-energy constraint required for the matched filter derivation, the pulse is normalized by dividing by the square root of its energy:

$$E_p = \sum |p[n]|^2 = 5^2 + 4^2 + 3^2 + 2^2 + 1^2 = 55 \quad \dots(\text{Eq. 3})$$

$$p_{\text{norm}}[n] = p[n] / \sqrt{55} \quad \dots(\text{Eq. 4})$$

Reference: Proakis & Salehi, 5th Ed., Section 6.2 — Unit-energy pulse normalization

A custom `normalize_power()` function, its code is shown in figure2, is used instead of any built-in normalization command, as required by the project specification. The function simply takes the array we want to normalize, calculates its power by summing the square of its elements then produces a new array having values equal to the old array divided by the square root of its total power.

```

192  [-]  function [array_normalized] = normalize_power(array)
193      array_power = 0;
194  [-]  for i = 1: length(array)
195      array_power = array_power + abs(array(i))^2;
196      end
197      array_normalized = array / sqrt(array_power);
198  end
199
200  [+]  
function [BER] = bit_error_rate (original, received) [...]
```

Figure 2: `normalize_power` Function

2.3 AWGN Channel Model

The channel adds zero-mean Gaussian noise $n[n]$ to the transmitted signal. The noise is generated with unit variance using `randn()` and scaled to achieve the desired two-sided power spectral density $N_0/2$:

$$n[n] = \sqrt{(N_0/2)} \cdot \text{randn}(\cdot) \quad \dots(\text{Eq. 5})$$

Reference: Haykin, 4th Ed., Eq. (6.66) — AWGN noise variance scaling

The received signal is therefore $v[n] = y[n] + n[n]$. The energy per bit is $E_b = \sum p^2[n] = 1$ (unit energy pulse), so the E_b/N_0 ratio in linear scale is simply $1/N_0$.

3. Requirement 1 — Matched Filter and Correlator (Noise-Free)

3.1 Matched Filter Design

The transmitted signal, as shown in line16, is simply the convolution between the binary polar signal we generated in line 11(upsampled version) and the pulse shaping function

The optimal linear filter for maximising the output SNR at the sampling instant in the presence of AWGN is the matched filter. Its impulse response is the time-reversed and delayed version of the pulse shape:

$$h_{MF}[n] = p[L - 1 - n] = \text{fliplr}(p) \dots (\text{Eq. 6})$$

Reference: Haykin, 4th Ed., Section 6.2 — Matched filter theorem: $h(t) = k \cdot p(T_s - t)$

This is implemented in MATLAB using the `fliplr()` command, shown in line 18, producing the reversed sequence [1, 2, 3, 4, 5]/ $\sqrt{55}$. The filtered signal is obtained by convolving the received signal with `h_MF`.

3.2 Non-Matched Filter (Filter 2)

For comparison, a rectangular (flat) filter of the same length $L = 5$, generated in line 23 figure 3 is also applied. This filter is not matched to $p[n]$ and therefore does not maximize the output SNR. Its normalized version is shown in line 24. The output of this filter at the sampling instants is expected to be lower in amplitude and more susceptible to errors compared to the matched filter.

```

23     h2 = ones(1, nspd);
24     h2_normalized = normalize_power(h2);
25     y_filtered2 = conv(y, h2_normalized);
26     y_filtered2_resampled = downsample(y_filtered2, nspd, nspd-1);

```

Figure 3: Second Filter Code

3.3 Sampling Instants

After convolution with a matched filter, the peak of the autocorrelation of $p[n]$ occurs at index $L = 5$. Since the pulse occupies exactly one symbol period, the zero-ISI condition of the Nyquist criterion is satisfied and the output is sampled every $L = 5$ samples starting from sample index L . A `downsample()` as shown in line 20 figure 1. We expressed it as simply the down sampling of `y_filtered1` with period T_s (`nspd` samples) at the end of each period (5th sample in each period), so `offset = 4` (for 1st sample `offset = 0`, so `offset = nspd - 1`). We used the same criteria for the second filter as shown in line 26 figure 3.

3.4 Correlator Receiver

The correlator receiver is mathematically equivalent to the matched filter at the sampling instants. For each symbol k , it multiplies the received signal over the k -th symbol interval by the pulse shape $p[n]$ and accumulates the result:

$$r_k = \sum_{n=0}^{L-1} y[k \cdot L + n] \cdot p[n] \dots (\text{Eq. 8})$$

Reference: Proakis & Salehi, 5th Ed., Eq. (6.4-8) — Correlator = matched filter at sampling instant

The correlator is implemented via a double loop as shown in figure 4: an outer loop iterates over each of the 10 symbols, at the beginning of which, the accumulator memory is reset (line 65), and an inner loop accumulates the running product, maintaining an accumulator memory between samples to replicate the integrate-and-dump behaviour shown in the project block diagram.

```
62 %Correlator
63 corr_out = zeros(1, nspd*no_of_binary_samples + 2*nspd -2);
64 for i=1: no_of_binary_samples
65     acc_mem = 0; %accumulator memory
66     for j = 1: nspd
67         corr_out((i-1)*nspd + j) = acc_mem + y((i-1) * nspd + j) * p_normalized(j);
68         acc_mem = corr_out((i-1)*nspd + j);
69     end
70 end
71
72 corr_out_resampled = downsample(corr_out, nspd, nspd-1);
```

Figure 4: Correlator Code

3.5 Results — Requirement 1a: Filter Output Comparison

Figure 5 shows the output of the matched filter (top, blue) and the non-matched rectangular filter (bottom, red) for a noise-free system with $N = 10$ bits. The black stems mark the values at the sampling instants. The matched filter output at the sampling instants equals $\pm E_b = \pm 1$ (for unit-energy pulse), while the rectangular filter produces smaller and unequal samples, confirming its suboptimal nature. Source code for plots is attached in figure 6. It is worth noting that the output sequence changes in each run due to the use of randi function. The new axis time mentioned in line29 figure 6 is sized to match the plots.

3.6

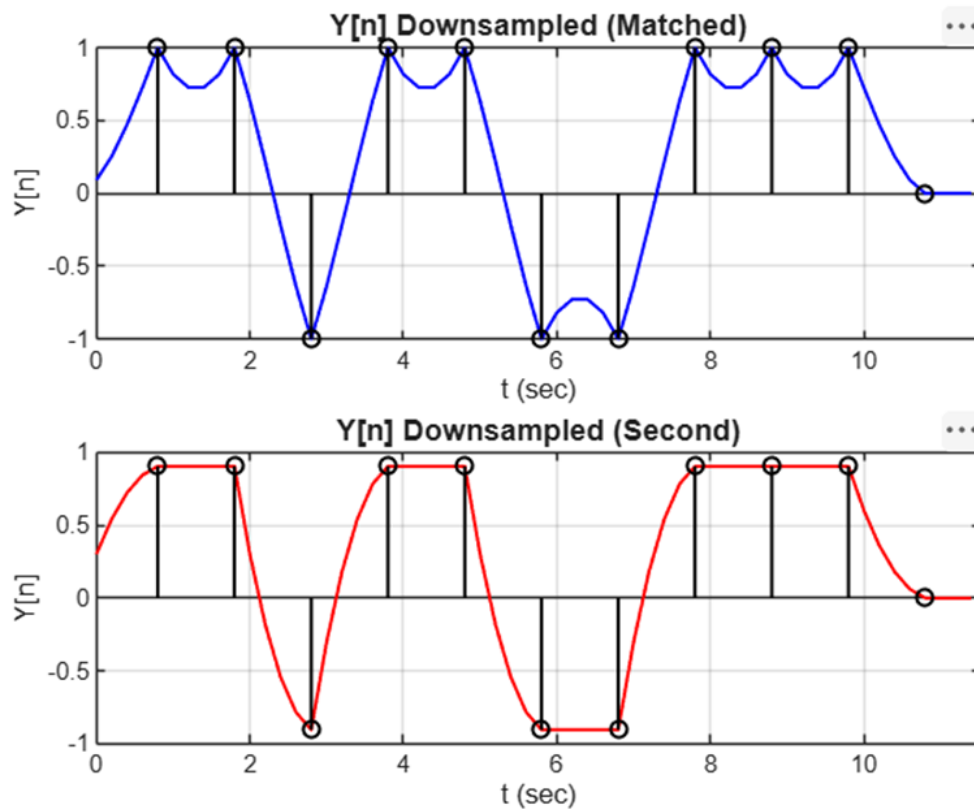


Figure 5: Top — Matched filter output $y_{MF}[n]$ (blue) with sampled values (black stems). Bottom — Rectangular filter output $y_{Rect}[n]$ (red) with sampled values (black stems). Noise-free, $N = 10$ bits.

```

28 %time axis for plotting
29 t_axis_new = 0: Ts/nspd: (nspd*no_of_binary_samples + 2*nspd -3) *(Ts/nspd);
30 t_axis_downsampled = downsample(t_axis_new, nspd, nspd-1);
31
32 figure('Name', 'Requirement1a', 'Color', [1 1 1]);
33
34 subplot(2, 1, 1);
35 plot(t_axis_new, y_filtered1, 'b', 'LineWidth', 1.2);
36 title('Y[n] Filtered by Matched Filter', 'FontSize', 11);
37 xlabel('t (sec)');
38 ylabel('Y[n]');
39 grid on;
40
41 hold on;
42 stem(t_axis_downsampled, y_filtered1_resampled, 'k', 'LineWidth', 1.2);
43 title('Y[n] Downsampled (Matched)', 'FontSize', 11);
44 xlabel('t (sec)');
45 ylabel('Y[n]');
46 grid on;
47
48 subplot(2, 1, 2);
49 plot(t_axis_new, y_filtered2, 'r', 'LineWidth', 1.2);
50 title('Y[n] Filtered by Second Filter', 'FontSize', 11);
51 xlabel('t (sec)');
52 ylabel('Y[n]');
53 grid on;
54
55 hold on;
56 stem(t_axis_downsampled, y_filtered2_resampled, 'k', 'LineWidth', 1.2);
57 title('Y[n] Downsampled (Second)', 'FontSize', 11);
58 xlabel('t (sec)');
59 ylabel('Y[n]');
60 grid on;

```

Figure 6: Requirement 1.a. Plots Source Code

3.6 Results — Requirement 1b: Matched Filter vs. Correlator

Figure 7 overlays the matched filter waveform with the correlator output and separately shows the sampled values of each. The samples of both receivers are numerically identical at every sampling instant, confirming the theoretical equivalence between the matched filter and the correlator for the rectangular pulse in a noise-free environment. Source code for the plot is given in figure 8.

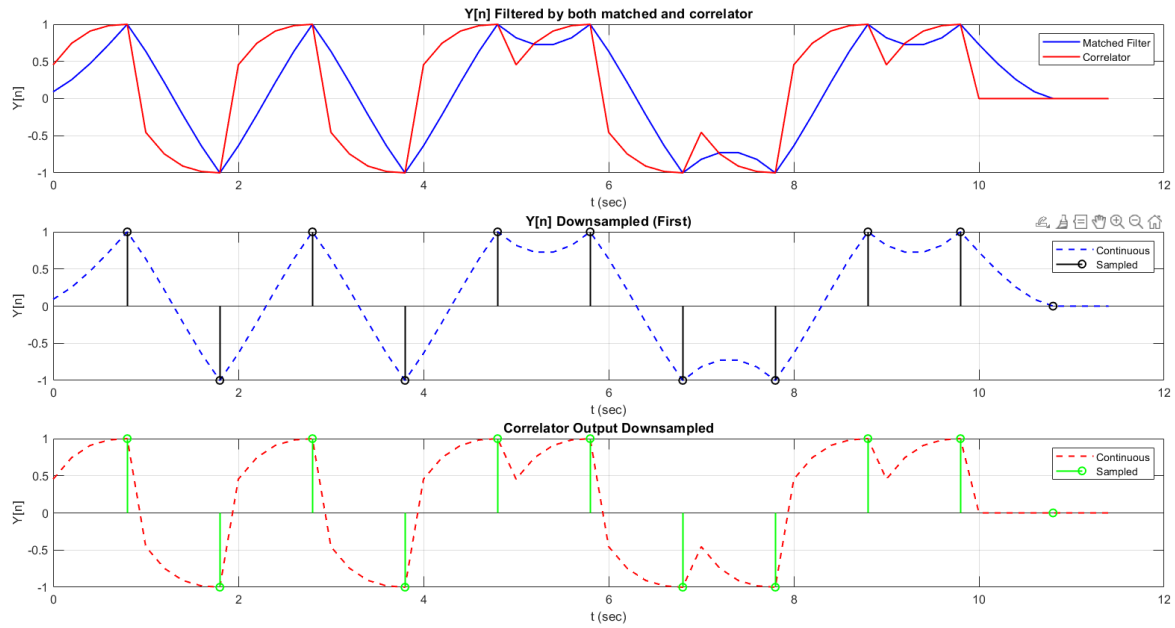


Figure 7: Blue — matched filter continuous output. Red — correlator running output. Stems — sampled values at intervals T_s . Both receivers yield identical samples.

```

74 figure('Name', 'Requirement1b', 'Color', [1 1 1]);
75 subplot(3, 1, 1);
76 plot(t_axis_new, y_filtered1, 'b', 'LineWidth', 1.2);
77 hold on;
78 plot(t_axis_new, corr_out, 'r', 'LineWidth', 1.2);
79 title('Y[n] Filtered by both matched and correlator', 'FontSize', 11);
80 xlabel('t (sec)');
81 ylabel('Y[n]');
82 legend('Matched Filter', 'Correlator');
83 grid on;
84 subplot(3, 1, 2);
85 plot(t_axis_new, y_filtered1, 'b', 'LineWidth', 1.2, 'LineStyle', '--');
86 hold on;
87 stem(t_axis_downsampled, y_filtered1_resampled, 'k', 'LineWidth', 1.2);
88 title('Y[n] Downsampled (First)', 'FontSize', 11);
89 xlabel('t (sec)');
90 ylabel('Y[n]');
91 legend('Continuous', 'Sampled');
92 grid on;
93 subplot(3, 1, 3);
94 plot(t_axis_new, corr_out, 'r', 'LineWidth', 1.2, 'LineStyle', '--');
95 hold on;
96 stem(t_axis_downsampled, corr_out_resampled, 'g', 'LineWidth', 1.2);
97 title('Correlator Output Downsampled', 'FontSize', 11);
98 xlabel('t (sec)');
99 ylabel('Y[n]');
100 legend('Continuous', 'Sampled');
101 grid on;

```

Figure 8: Requirement 1.b. Plot Source Code

4. Requirement 2 — Noise Analysis and BER

4.1 Simulation Setup

The noise analysis uses $N = 10,000$ randomly generated bits to obtain statistically reliable BER estimates. As shown in line 104 figure 9. Actually lines from 104 to 107 are copied from line 8 to 11 figure 1; the value of the variable `no_of_binary_samples` is updated then `X` and binary polar signal are overwritten for forward analysis. For each value of E_b/N_0 in the range $[-2, +5]$ dB (1 dB steps), a new independent noise sequence is generated inside a for loop (line 113), added to the transmitted signal, and the receiver output is sampled.

The N_0 value for each E_b/N_0 point is derived from:

$$N_0 = Eb / (Eb/N_0)_{linear} = 1 / 10^{(Eb/N_0_{dB} / 10)} \dots (\text{Eq. 9})$$

The noise variance is then set to $N_0/2$ as per Equation 5. Both the matched filter receiver and the non-matched rectangular filter receiver are evaluated at each SNR point.

The for loop first converts each db N_0 value to a linear scale as shown in lines 115, 116. The theoretical value for each case is calculated, then noise signal is generated at line 119 then added to the transmitted signal then filtered by the matched filter and resampled. The same steps are followed for the square pulse case.

```

103 %% Requirement 2
104 no_of_binary_samples = 10000;
105 x = randi([0 1], 1, no_of_binary_samples);
106 bps = 2*x-1; %binary polar signal
107 bps = upsample (bps, nspd);
108 y = conv (bps, p_normalized);
109
110 BER_theoritcal = zeros (1, 8);
111 BER_practical_matched_filter = zeros (1, 8);
112 BER_practical_filter2 = zeros (1,8);
113 for db_sweep = -2: 1: 5
114
115     N0_db = 0 - db_sweep;
116     N0_linear = 10 ^ (N0_db/10);
117
118     BER_theoritcal(db_sweep + 3) = 0.5 *erfc(sqrt(1/N0_linear));
119     noise = sqrt (N0_linear/2) .* randn (size (y));
120     v = y + noise;
121     y_filtered1 = conv (v, h_matched);
122     y_filtered1_resampled = downsample (y_filtered1, nspd, nspd-1);
123     t_axis_new = 0: Ts/nspd: (nspd*no_of_binary_samples + 2*nspd -3) *(Ts/nspd);
124
125     %BER:
126     BER_practical_matched_filter(db_sweep + 3) = bit_error_rate ...
127         (x, y_filtered1_resampled);
128     h2 = 5 * ones(1, nspd);
129     y_filtered2 = conv(v, h2);
130     y_filtered2_resampled = downsample(y_filtered2, nspd, nspd-1);
131     BER_practical_filter2(db_sweep + 3) = bit_error_rate ...
132         (x, y_filtered2_resampled);
133 end

```

Figure 9: Requirement 2 Source Code

4.2 Hard Decision and BER Calculation

A custom function `bit_error_rate()`, its code is in figure 10, counts the fraction of incorrectly detected bits. The function receives two input arrays representing the original message and the received one (line 202 figure 10). It passes through a for loop of number of iterations = the length of the original message. A variable `received_binary` is declared as bit '1' if the received signal ≥ 0 , and bit '0' otherwise.

The decoded bit is then compared against the original transmitted bit, then if unequal the count for error bits, initialized to zero before the loop at line 204, will be incremented. Finally, the bit error rate will be outputted from the function from the following equation as shown in line 215:

$$BER = (\text{number of bit errors}) / (\text{total bits}) \dots (\text{Eq. 10})$$

```

202  function [BER] = bit_error_rate (original, received)
203      no_of_bits = length (original);
204      error_bits_count = 0;
205      for i = 1: no_of_bits
206          if received(i) >= 0
207              received_binary = 1;
208          else
209              received_binary = 0;
210          end
211          if received_binary ~= original(i)
212              error_bits_count = error_bits_count + 1;
213          end
214      end
215      BER = error_bits_count / no_of_bits;
216
217  end

```

Figure 10: Function `bit_error_rate` Source Code

4.3 Theoretical BER

For binary polar PAM transmitted through an AWGN channel and received by an ideal matched filter, the theoretical BER is given by:

$$BER_{theory} = (1/2) \cdot \text{erfc}(\sqrt{Eb/N_0}) \dots (\text{Eq. 11})$$

Reference: Haykin, 4th Ed., Eq. (6.69); Proakis & Salehi, 5th Ed., Eq. (6.4-25)

This result follows from the Gaussian distribution of the noise at the matched filter output. The signal component is $\pm E_b = \pm 1$, leading to the Q-function argument $\sqrt{(1/N_0)}$. Using the identity $Q(x) = (1/2)\text{erfc}(x/\sqrt{2})$ as shown in line 118 figure 10.

4.4 Results — BER vs E_b/N_0

Figure 11 shows the simulated BER for both the matched filter and the rectangular filter receivers, alongside the theoretical curve, plotted on a linear scale against E_b/N_0 in dB.

The matched filter simulated BER closely tracks the theoretical curve, validating the correctness of the simulation. The rectangular filter exhibits a higher BER at every SNR point, demonstrating the SNR penalty incurred by using a suboptimal receiver. The gap between the two simulated curves represents the degradation in performance caused by the mismatch between the filter and the pulse shape. The source code for this plot is given in figure 12

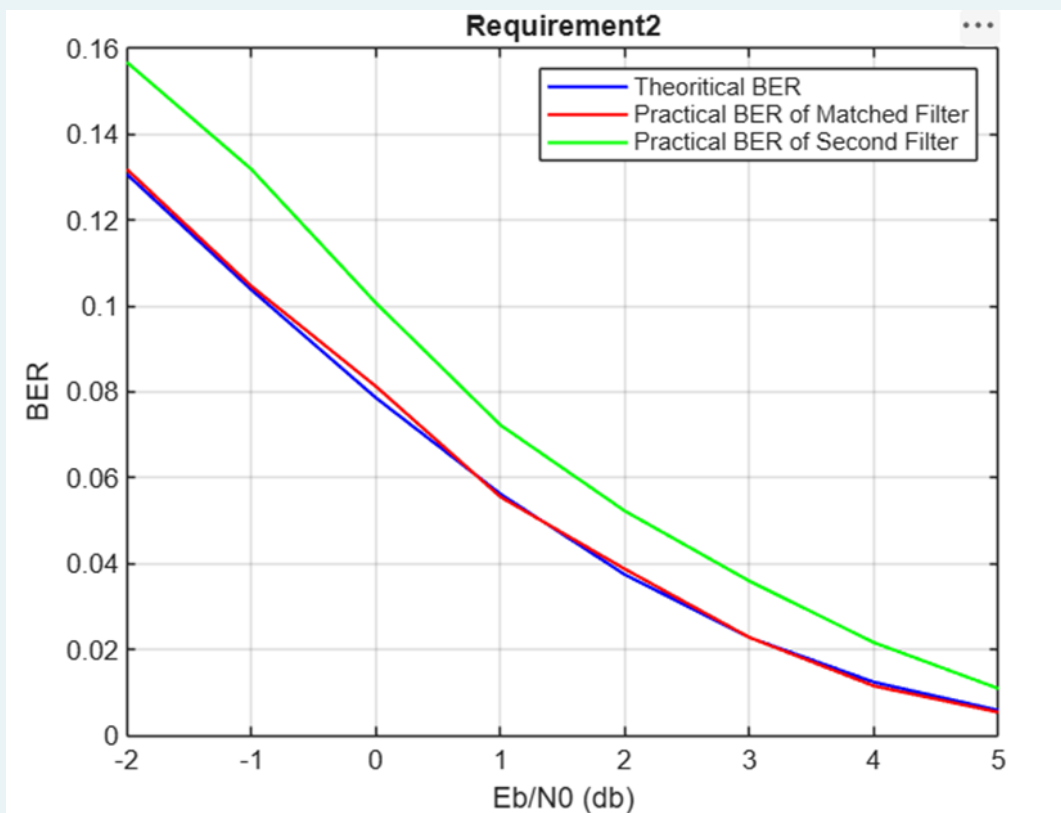


Figure 11: BER versus E_b/N_0 (dB). Blue — theoretical BER = $0.5 \cdot \text{erfc}(\sqrt{E_b/N_0})$. Red — simulated matched filter. Green — simulated rectangular filter. $N = 10,000$ bits

```

135 x_axis_of_BER = -2:1:5;
136 figure('Name', 'Requirement2', 'Color', [1 1 1]);
137 plot (x_axis_of_BER, BER_theoretical, 'b', 'LineWidth', 1.2);
138 hold on;
139 plot (x_axis_of_BER, BER_practical_matched_filter, 'r', 'LineWidth', 1.2);
140 hold on;
141 plot (x_axis_of_BER, BER_practical_filter2, 'g', 'LineWidth', 1.2);
142 title('Requirement2', 'FontSize', 11);
143 xlabel ('Eb/N0 (db)');
144 ylabel('BER');
145 legend ('Theoretical BER', 'Practical BER of Matched Filter', ...
146         'Practical BER of Second Filter');
147 grid on;

```

Figure 12: Requirement 2 Plot Source Code

5. Requirement 3 — ISI and Eye Patterns

5.1 Inter-Symbol Interference (ISI)

Inter-Symbol Interference occurs when the impulse response of the channel or filtering system extends beyond one symbol period, causing energy from one symbol to leak into adjacent symbol intervals. This degrades the decision at the receiver and increases the probability of error.

The Nyquist condition for zero ISI states that the combined impulse response of transmit filter, channel, and receive filter must have zero crossings at all non-zero multiples of the symbol period T_s . The Raised Cosine (RC) spectrum satisfies this condition exactly.

Reference: Proakis & Salehi, 5th Ed., Section 9.2 — Nyquist criterion for zero ISI

5.2 Square-Root Raised Cosine Filtering

Rather than applying the full Raised Cosine filter at the transmitter or receiver alone, the RC response is split equally between the transmit (Tx) and receive (Rx) filters. Each filter has a Square-Root Raised Cosine (SRRC) spectrum, so that their combined response satisfies:

$$H_{Tx}(f) \cdot H_{Rx}(f) = H_{RC}(f) \quad \dots(\text{Eq. 12})$$

Reference: Proakis & Salehi, 5th Ed., Eq. (9.2-24) — SRRC filter pair

This split design is standard in practice because it distributes the pulse-shaping responsibility between both ends of the link and makes the receive filter also act as a matched filter to the transmitted pulse, simultaneously achieving zero ISI and maximum SNR.

5.3 Filter Parameters

The SRRC filter is generated using the `rcosdesign()` command with parameters $F_d = 1/T_s = 1$ Hz (symbol rate) and $F_s = L/T_s = 5$ Hz (sample rate). The roll off factor R and the delay (in symbol periods) are varied across four cases:

Case	Rolloff (R)	Delay (symbols)	Expected Eye at B
1	0	2	Narrow / partially closed
2	0	8	Wider
3	1	2	Wide open
4	1	8	Widest and cleanest opening

5.4 Eye Pattern Interpretation

Generally, the vertical eye opening is maximum at the center of the eye meaning that this is the best time for sampling.

The eye pattern is constructed by overlaying successive segments of the received waveform, each of duration equal to one symbol period (one eye opening). A wide, clearly open eye at the sampling instant (the center of the eye) indicates low ISI and reliable detection. A closed or nearly closed eye indicates severe ISI.

The following observations apply to the four cases:

- Point A (after Tx filter only): the eye is partially or fully closed because only one SRRC filter has been applied. The combined response SRRC is not the full Raised Cosine, so Nyquist's ISI criterion is not met.

- Point B (after both Tx and Rx filters): the eye opens because the combined response $\text{SRRC} \times \text{SRRC} = \text{RC}$ satisfies the Nyquist criterion. The optimal sampling instant coincides with the center of the open eye.
- $R = 0$, delay = 2 (Case 1): minimum roll off, but short filter length is a poor approximation to the infinite ideal impulse response. The eye at B is narrow.
- $R = 0$, delay = 8 (Case 2): with a longer filter the approximation improves. The eye opens more clearly.
- $R = 1$, delay = 2 (Case 3): maximum roll off doubles the bandwidth but makes the pulse compact. The eye opens even for short delay.
- $R = 1$, delay = 8 (Case 4): the widest and cleanest eye opening, combining maximum roll off with a long filter for the best Nyquist approximation.

5.5 Results — Eye Patterns

Figures 4 through 11 show the eye diagrams at points A and B for all four filter cases.

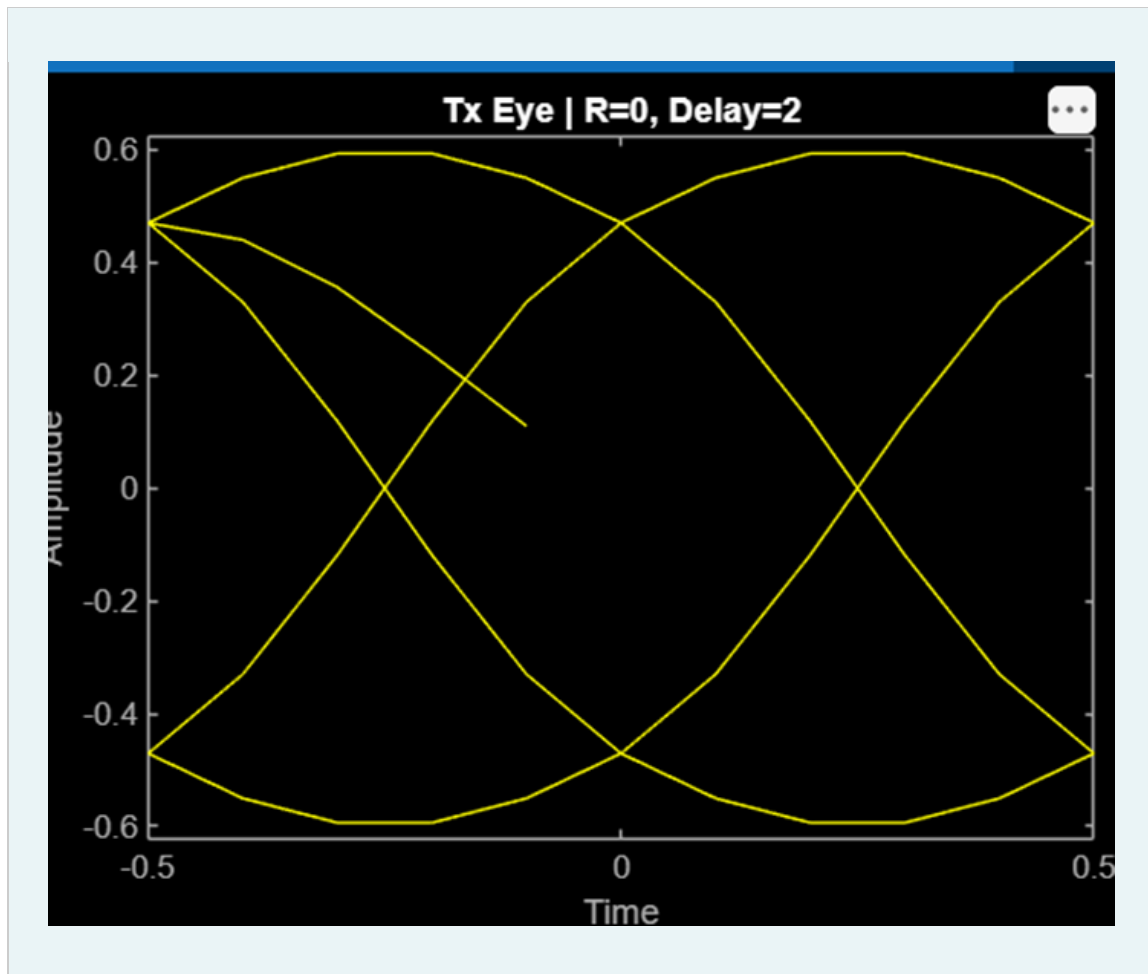


Figure 13: Eye pattern at point A (after Tx SRRC filter only). $R = 0$, delay = 2. Eye is partially closed due to ISI.

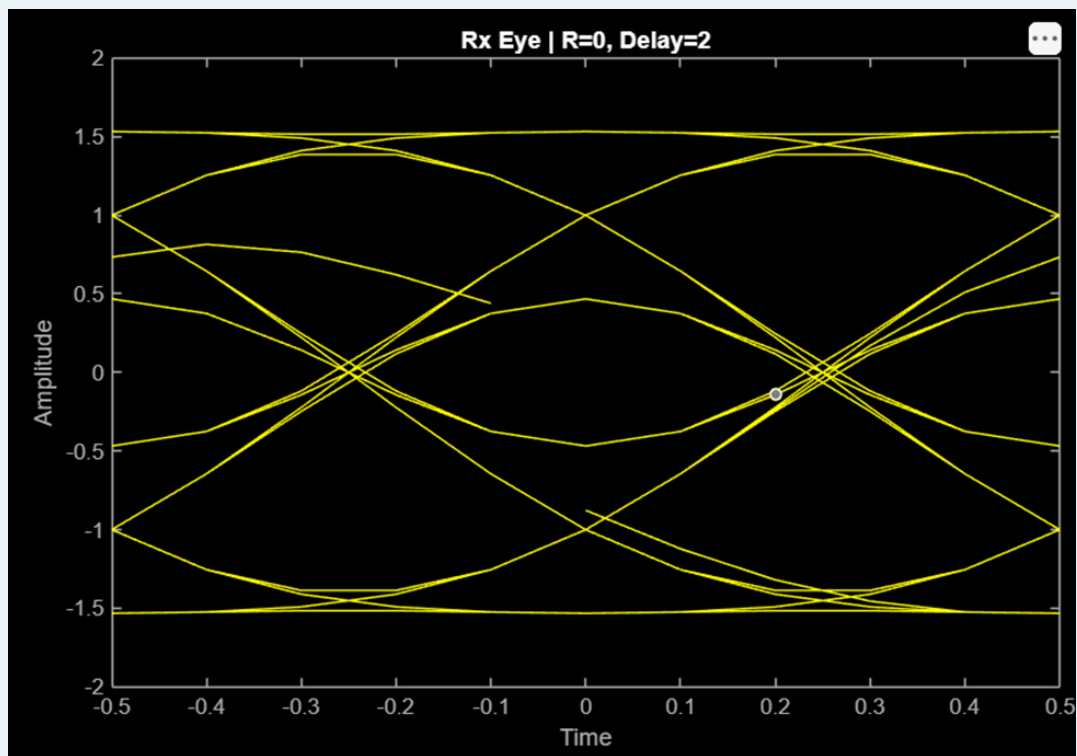


Figure 14: Eye pattern at point B (after Tx and Rx SRRC filters, combined RC response). $R = 0$, delay = 2. Eye opens but is narrow.

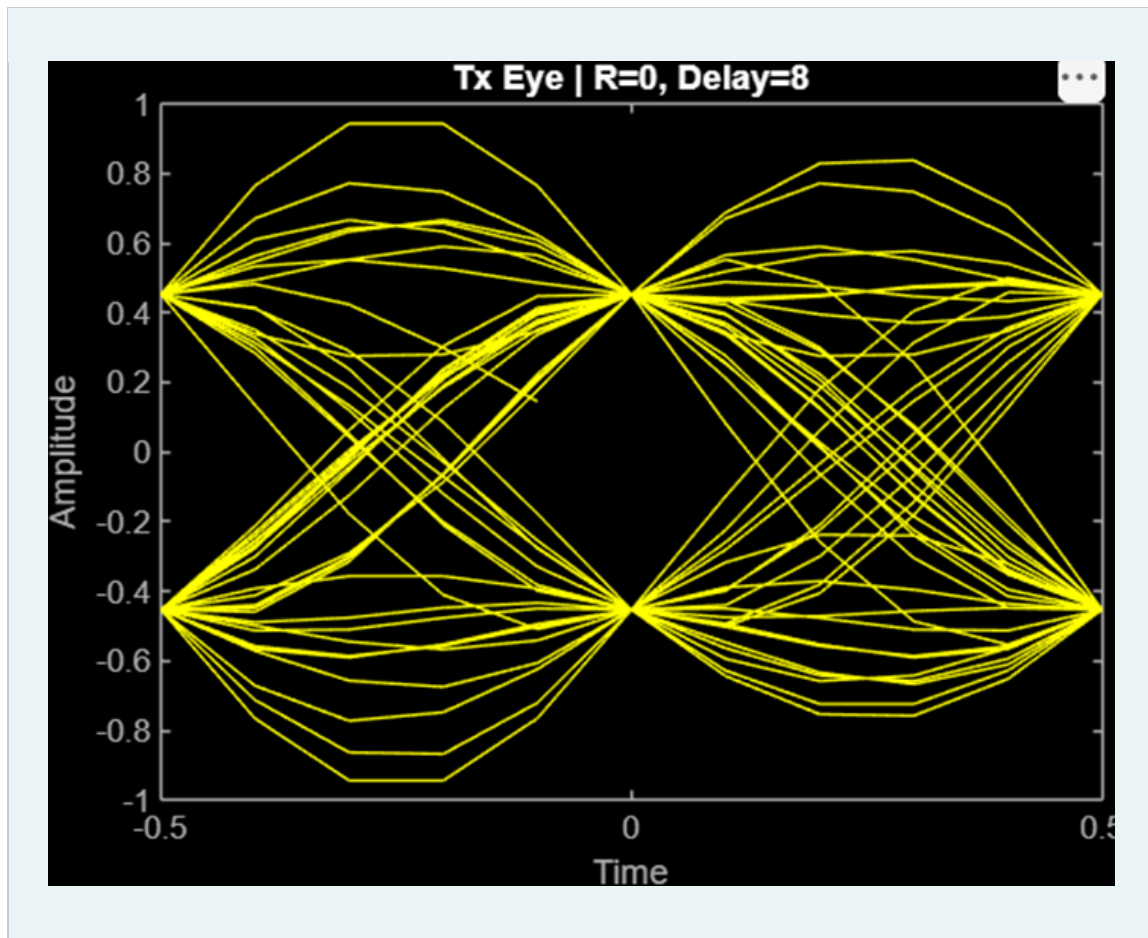


Figure 15: Eye pattern at point A. $R = 0$, delay = 8. ISI still present (single SRRC).

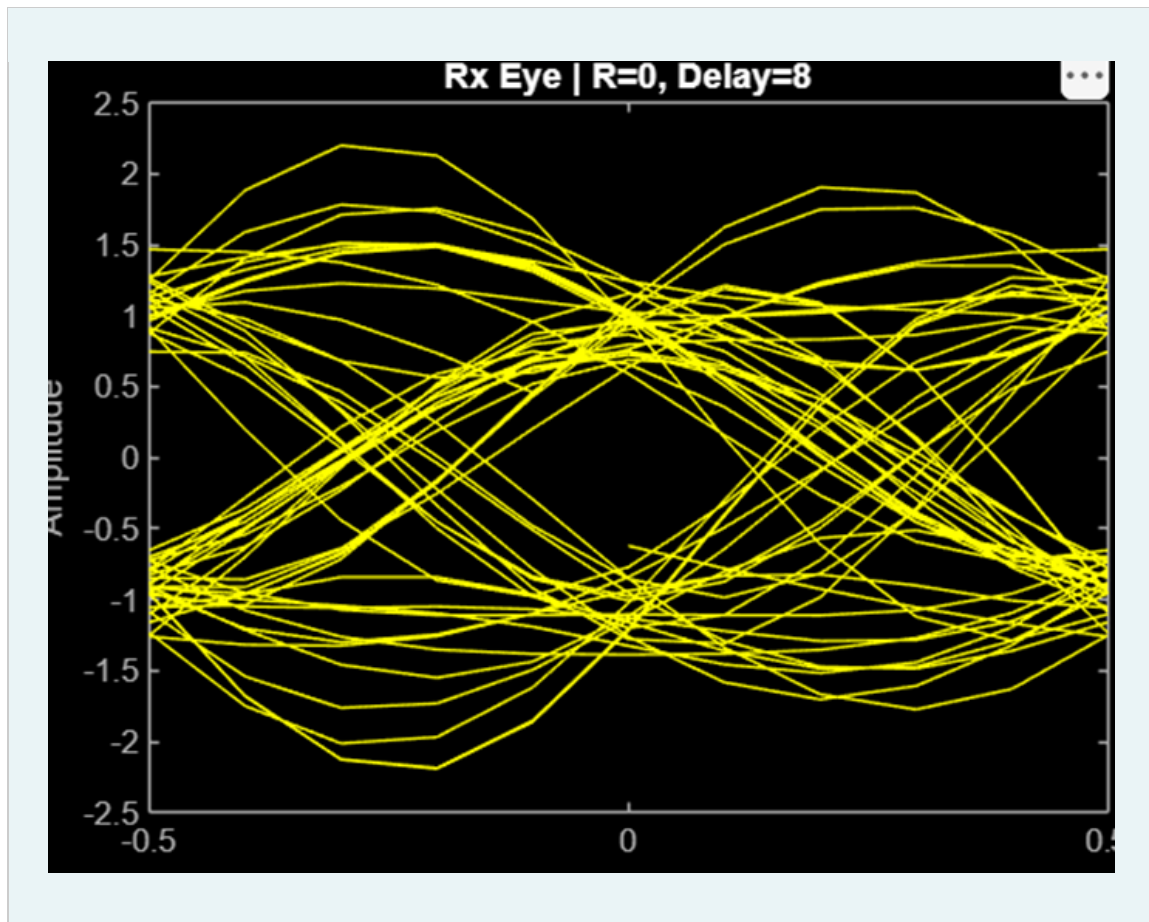


Figure 16: Eye pattern at point B. $R = 0$, delay = 8. Wider eye opening compared to delay = 2.

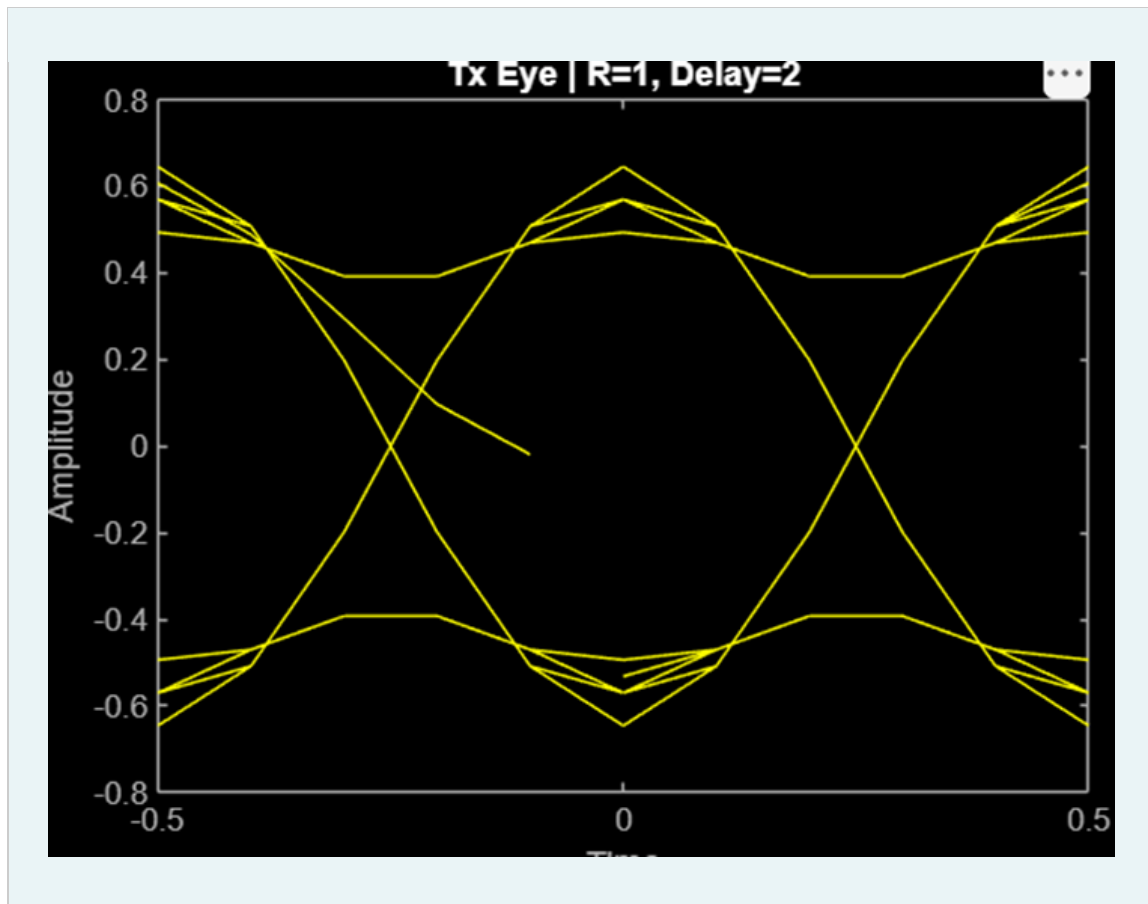


Figure 17: Eye pattern at point A. $R = 1$, delay = 2.

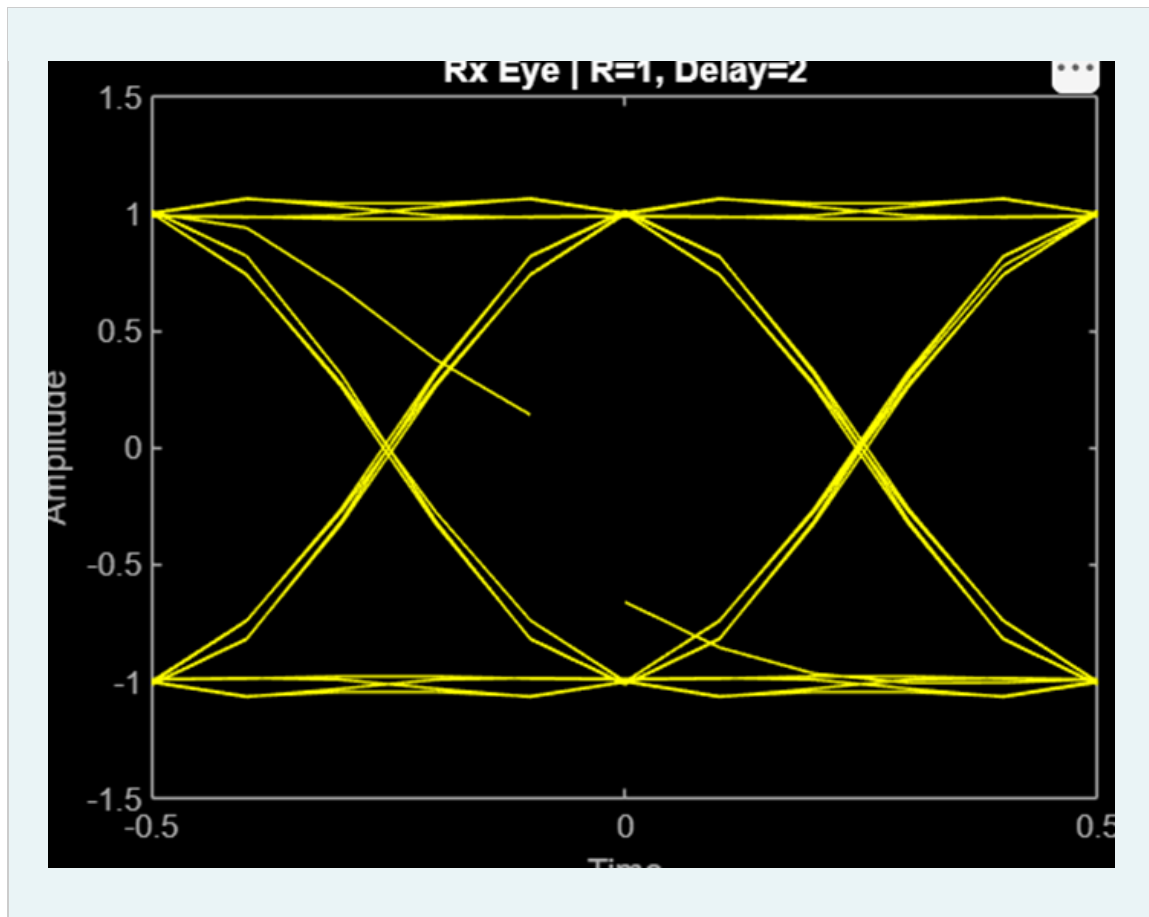


Figure 18: Eye pattern at point B. $R = 1$, delay = 2. Eye opens clearly due to high rolloff.

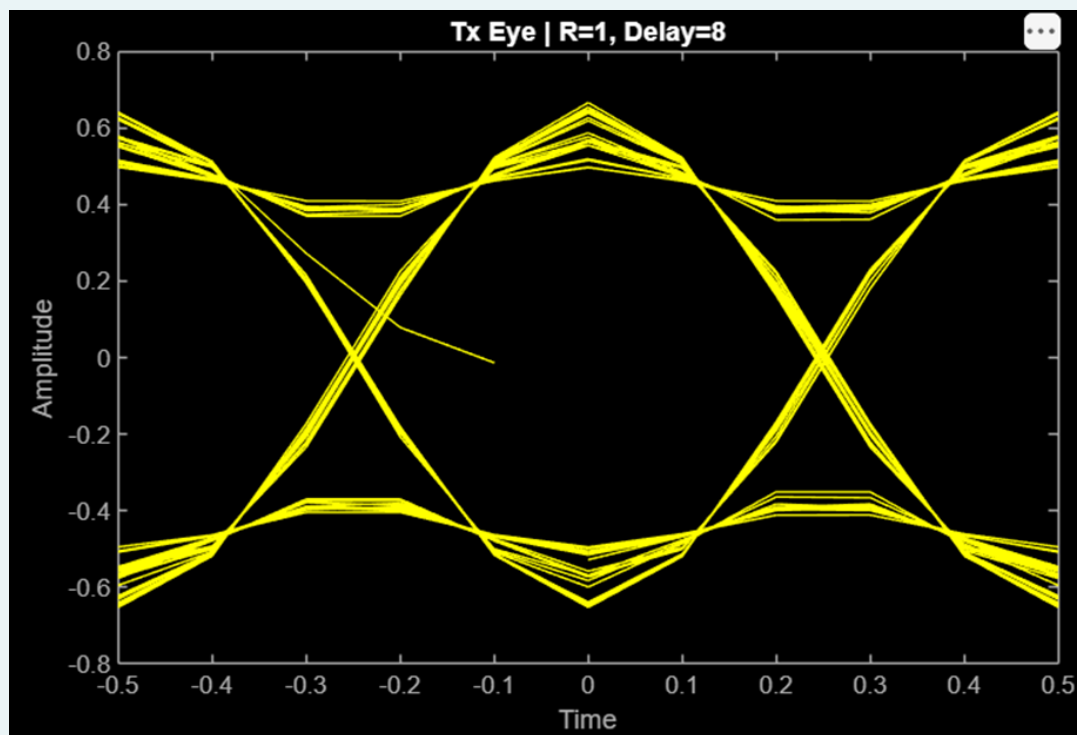


Figure 19: Eye pattern at point A. $R = 1$, delay = 8.

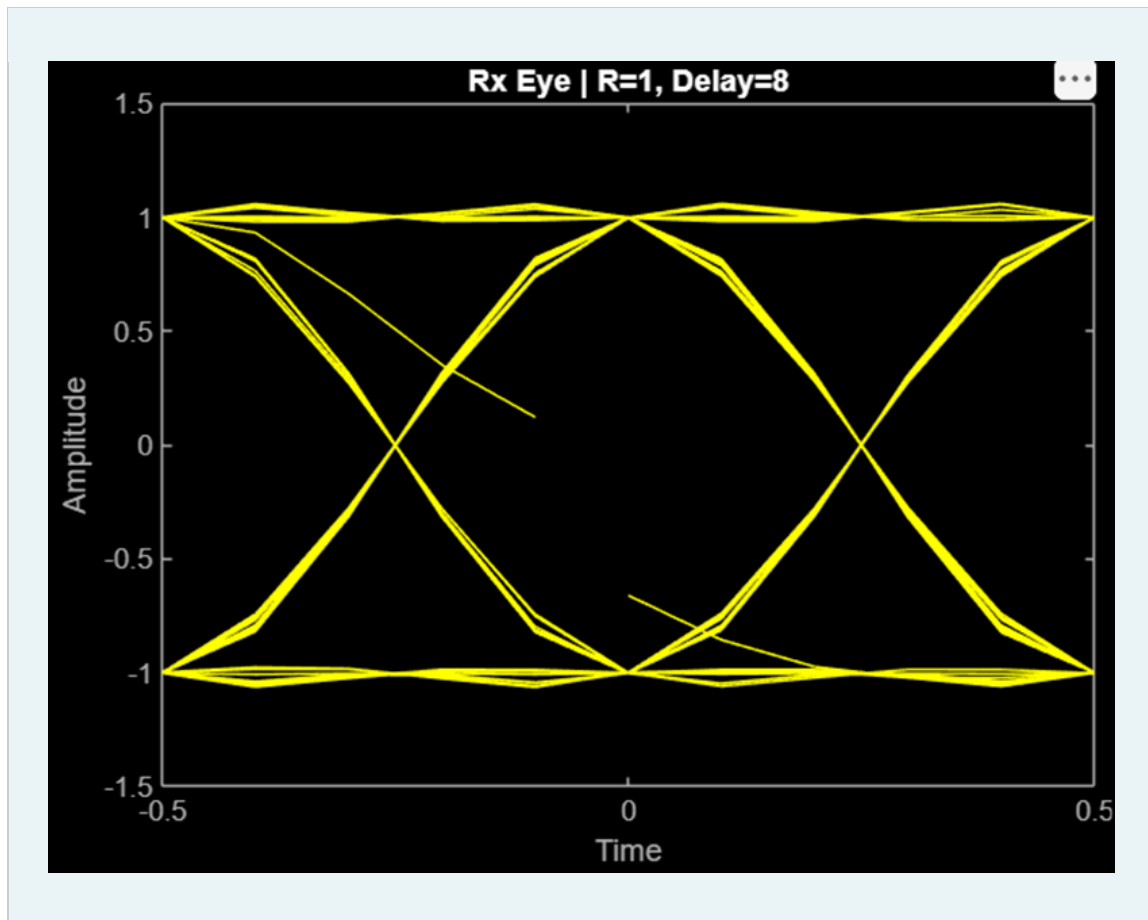


Figure 20: Eye pattern at point B. $R = 1$, delay = 8. Widest and most open eye across all four cases.

6. MATLAB Implementation

The MATLAB code is organized into three clear sections, one per requirement, followed by two custom helper functions explained through the report. The key implementation choices are summarised below.

The `normalize_power()` function manually computes the signal energy as $\sum |x[n]|^2$ and divides every sample by the square root of this energy, avoiding any built-in normalisation command.

The `bit_error_rate()` function iterates over all N bits, applies the threshold decision rule (sample $\geq 0 \rightarrow$ bit '1'), and returns the fraction of incorrectly decoded bits.

No built-in functions beyond those listed in the project specification are used. In particular, the correlator is implemented using explicit for-loops with an accumulator to replicate the integrate-and-dump operation.

7. Conclusion

This project successfully implemented and analysed three fundamental concepts in digital communications: matched filter design, BER performance in AWGN, and ISI mitigation using raised cosine pulse shaping.

The key findings are:

- The matched filter, defined as $h[n] = \text{fliplr}(p[n])$, maximises the output SNR at the sampling instant. Its samples are numerically identical to those of the correlator receiver, confirming the theoretical equivalence of the two structures.
- The non-matched rectangular filter yields lower sample amplitudes and higher BER at all SNR values, demonstrating the penalty of suboptimal receiver design.
- The simulated BER of the matched filter receiver closely follows the theoretical curve $\text{BER} = 0.5 \cdot \text{erfc}(\sqrt{E_b/N_0})$ across all tested E_b/N_0 values, validating the simulation.
- At point A (after only the Tx SRRC filter), the eye pattern is partially or fully closed due to ISI. At point B (after both SRRC filters whose combined response equals the full RC filter), the eye opens, confirming that the Nyquist condition for zero ISI is satisfied.
- Increasing the filter delay from 2 to 8 symbol periods improves the approximation to the ideal infinite-length SRRC filter, leading to a wider and cleaner eye opening. A higher rolloff factor $R = 1$ further widens the eye at the cost of increased bandwidth.

8. References

- [1] Haykin, S., "Communication Systems", 4th Edition, John Wiley & Sons, 2001. — Matched filter (Section 6.2), noise variance (Eq. 6.66), theoretical BER (Eq. 6.69).
- [2] Proakis, J.G. & Salehi, M., "Digital Communications", 5th Edition, McGraw-Hill, 2008. — Polar mapping (Eq. 1.1-6), unit-energy normalisation (Section 6.2), theoretical BER (Eq. 6.4-25), Nyquist ISI criterion (Section 9.2), SRRC filter pair (Eq. 9.2-24).
- [3] Dr. Mohamed Nafie, Lecture Slides — Introduction to Communications, Cairo University, 2025/2026.

